

ASR Data Quality Classifier

Automatically classify audio + transcript pairs as usable / not-usable for ASR training

Issued	2026-06-01
Deadline	TBD — confirm with supervisor
Deliverables	Code + trained model uploaded to your blob folder
Data location	Azure Blob Storage (token provided separately)

1. Background

We are building an Automatic Speech Recognition (ASR) system. Before feeding data into the ASR training pipeline, each sample — an audio recording paired with its text transcript — must be verified as **usable**: the audio should be clean and intelligible, and the transcript should accurately match what is spoken.

Previously, interns listened to each audio file and read the transcript manually to decide whether the pair was acceptable. This manual process does not scale. Your task is to build a machine-learning classifier that automates this quality-control step.

2. Dataset

The labeled dataset is stored in Azure Blob Storage. You will receive a **SAS token** with read access. The dataset contains:

- **~4,000 samples**, each consisting of one audio file and one transcript file.
- A **label file** (CSV/JSON) produced by manual annotation: **1 = usable** (audio and transcript match, acceptable quality), **0 = not usable** (mismatch, noisy audio, wrong transcript, etc.).
- The split between usable and not-usable may be **imbalanced** — check this first.

Expected blob container structure:

Path	Description
data/audio/*.wav (or .mp3)	Raw audio files
data/transcripts/*.txt	Corresponding transcript files
data/labels.csv	Columns: file_id, label (0/1)
/	Your output folder — write all deliverables here

Note: exact folder names will be confirmed when the SAS token is shared.

3. Task Description

Build a binary classifier that takes an **audio file** and its **transcript** as input and outputs a label: **1 (usable)** or **0 (not usable)**. The classifier should be accurate enough to replace manual listening for the bulk of the data.

3.1 Suggested Approach

You are free to design the system as you see fit. The following is a recommended starting point, not a strict requirement:

- 1 **Exploratory Data Analysis (EDA)** — inspect label distribution, audio lengths, transcript lengths, and sample a few examples to understand what makes a sample unusable.
- 2 **Feature extraction**
 - *Audio features*: MFCCs, log-Mel spectrogram, pitch/energy statistics, SNR estimate, silence ratio, or embeddings from a pretrained model (e.g. Wav2Vec2, Whisper encoder).
 - *Transcript features*: length, OOV rate, language-model perplexity, or embeddings (e.g. sentence-transformers).
 - *Cross-modal features*: ASR-hypothesis vs. provided transcript similarity (e.g. WER/CER, cosine similarity of embeddings).
- 3 **Model selection** — start simple (logistic regression, gradient boosting on hand-crafted features) before moving to deep learning. Pick what achieves the best F1/AUC on the validation set.
- 4 **Evaluation** — use stratified k-fold or a fixed train/val/test split. Report precision, recall, F1-score, and ROC-AUC. Pay attention to the minority class.
- 5 **Error analysis** — inspect misclassified samples and document findings.

3.2 What You Must NOT Do

- Do **not** use the test set for any hyperparameter tuning or model selection.
- Do **not** hard-code the SAS token or any credential inside the code. Read it from an environment variable or a config file that is **not** committed.
- Do **not** retrain on the full dataset without a held-out evaluation — you must be able to reproduce the reported metrics.

4. Deliverables

Upload **all** of the following to your personal folder in the blob container (<your_name>/):

#	Item	Details
1	Source code	Well-structured Python project. Include a README.md with setup and run instructions.
2	Trained model file(s)	Saved model weights / serialized pipeline (e.g. .pt, .pkl, .onnx).
3	Evaluation report	A short report (PDF or Markdown) with metrics table, confusion matrix, and error analysis.
4	requirements.txt (or environment.yml)	Exact package versions so the code can be reproduced.

5	Inference script	A standalone script that loads the model and classifies a given audio+transcript pair.
---	------------------	--

5. Evaluation Criteria

Your work will be reviewed on the following dimensions:

Criterion	Weight	What we look for
Model performance	40%	F1-score (macro) and ROC-AUC on the held-out test set. A reasonable baseline is $F1 \geq 0.80$.
Code quality	25%	Readable, modular, documented code. No hard-coded paths or secrets.
Reproducibility	20%	Can the model be retrained from scratch using your code and the provided data?
Report & analysis	15%	Clear explanation of your choices, honest error analysis, and lessons learned.

6. Technical Requirements

- **Language:** Python 3.9+
- **Azure SDK:** use azure-storage-blob to download data and upload results. Authenticate using the provided SAS token (read from environment variable AZURE_SAS_TOKEN).
- **Version control:** keep your code in a git repository. Commit regularly with meaningful messages. You may use a private GitHub/GitLab repo during development; final code must be uploaded to the blob.
- **Open-source libraries are allowed** — no proprietary or licensed-for-internal-use-only models unless approved.

7. Submission

- 1 Upload all deliverables to **<your_name>/** in the blob container.
- 2 Send a brief email / message to your supervisor with:
 - (a) confirmation that upload is complete,
 - (b) your best validation F1-score and ROC-AUC,
 - (c) a one-paragraph summary of your approach.

Questions? Reach out to your supervisor before spending more than 30 minutes stuck on any single issue.

Good luck! — ASR Team